Deliverable A: Corrected Implementation and Runnable Tests

The complete, self-contained implementation (pigou_j_core.py) and executable unit-test suite (test_pigou_j_core.py) are provided below as separate modules, followed by the actual execution report from the local Python environment.

A.1 File 1: pigou_j_core.py (Corrected Core Implementation)

```
import math
import random
from dataclasses import dataclass
from typing import List, Dict, Tuple, Optional

BLOCK_SIZE_TOKENS = 16


@dataclass
class RingSlot:
    step_id: int
    resolved: bool
    err_hat: float


@dataclass
class IncumbentSeq:
    req_id: int
    prompt_tokens: int
    private_kv_tokens: int
    elapsed_decode_tokens: int
    pred_total_decode_tokens: int

    @property
    def remaining_decode_tokens(self) -> int:
        return max(0, self.pred_total_decode_tokens - self.elapsed_decode_tokens)

    @property
    def logical_context_tokens(self) -> int:
        return self.prompt_tokens + self.elapsed_decode_tokens


class VerifiedIWDACIGate:
    """
    Implements Theorem 2 with zero-based arrival indexing t = 0, 1, 2, ...,
    tagged ring-buffer slots, timeout handling at lag h, and advancement on
    every arrival (including held or rejected requests).
    """
    def __init__(self, lag_h: int, alpha_target: float, gamma: float, p_audit: float, cal_scores:
```

```
List[float]):
        self.h = max(1, int(lag_h))
        self.alpha_target = float(alpha_target)
        self.gamma = float(gamma)
        self.p_audit = float(p_audit)
        self.alpha_t = float(alpha_target)
        self.cal_scores = sorted(float(s) for s in cal_scores)
        self.tau_t = self._compute_tau(self.alpha_t)
        self.step_idx = 0
        # Pre-populate ring buffer for steps s in {-h, ..., -1} with neutral error alpha_target
        self.buf = [
            RingSlot(step_id=-self.h + idx, resolved=True, err_hat=self.alpha_target)
            for idx in range(self.h)
        ]

    def _compute_tau(self, alpha_val: float) -> float:
        if alpha_val <= 0.0 or not self.cal_scores:
            return float("inf")
        if alpha_val >= 1.0:
            return float("-inf")
        q = 1.0 - alpha_val
        n = len(self.cal_scores)
        k = int(math.ceil((n + 1) * q))
        if k > n:
            return float("inf")
        return self.cal_scores[max(0, k - 1)]

    def check_gate_eligibility(self, s_hat: float, domain_C: float, C_max: float) -> bool:
        if math.isnan(s_hat) or math.isnan(domain_C):
            return False
        return (self.alpha_t > 0.0) and (domain_C <= C_max) and (s_hat >= self.tau_t)

    def commit_arrival_and_step(self, routed_to_small: bool, rng: random.Random = random) ->
Tuple[int, bool]:
        t_cur = self.step_idx
        slot_idx = t_cur % self.h
        self.step_idx += 1

        # 1. Read delayed feedback from arrival (t_cur - h) before overwriting slot_idx
        old_slot = self.buf[slot_idx]
        err_hat_delayed = old_slot.err_hat if old_slot.resolved else (1.0 / self.p_audit)

        # 2. Write current arrival t_cur into slot_idx
        audit_triggered = False
        if not routed_to_small:
            # Routed to LARGE or held/rejected: comparative error is deterministically 0.0
            self.buf[slot_idx] = RingSlot(step_id=t_cur, resolved=True, err_hat=0.0)
        else:
```

```
            if rng.random() < self.p_audit:
                audit_triggered = True
                # Pending audit; defaults to conservative 1 / p_audit if unresolved at t_cur + h
                self.buf[slot_idx] = RingSlot(step_id=t_cur, resolved=False, err_hat=1.0 / self.p_audit)
            else:
                self.buf[slot_idx] = RingSlot(step_id=t_cur, resolved=True, err_hat=0.0)

        # 3. Update alpha_{t+1} and tau_{t+1} for the next arrival
        self.alpha_t = min(1.0, self.alpha_t + self.gamma * (self.alpha_target - err_hat_delayed))
        self.tau_t = self._compute_tau(self.alpha_t)
        return t_cur, audit_triggered

    def resolve_shadow_audit(self, arrival_step_id: int, small_correct: bool, large_correct: bool)
-> bool:
        slot_idx = arrival_step_id % self.h
        slot = self.buf[slot_idx]
        if slot.step_id == arrival_step_id and not slot.resolved:
            err_real = 1.0 if (not small_correct and large_correct) else 0.0
            slot.err_hat = err_real / self.p_audit
            slot.resolved = True
            return True
        return False


class MockWorker:
    def __init__(self, worker_id: int, model: str, tier: str, max_kv_tokens: int = 16384, chunk_size:
int = 512, is_disagg: bool = False):
        self.id = worker_id
        self.model = model
        self.tier = tier
        self.max_kv_tokens = max_kv_tokens
        self.chunk_size = chunk_size
        self.chunk_mix = True
        self.is_disagg = is_disagg
        self.incumbents: List[IncumbentSeq] = []
        self.queued_prefill_tokens: int = 0
        self.phys_kv_tokens: int = 0
        self.attn_kv_tokens: int = 0

    @property
    def free_kv_ratio(self) -> float:
        return max(0.0, 1.0 - (self.phys_kv_tokens / float(self.max_kv_tokens)))

    def record_inflight(self, p_t: int, p_eff: int, d_hat: int):
        new_phys = BLOCK_SIZE_TOKENS * math.ceil((p_eff + d_hat) / BLOCK_SIZE_TOKENS)
        self.phys_kv_tokens += new_phys
        self.attn_kv_tokens += (p_t + d_hat)
        new_inc = IncumbentSeq(
```

```
            req_id=1000 + len(self.incumbents),
            prompt_tokens=p_t,
            private_kv_tokens=new_phys,
            elapsed_decode_tokens=0,
            pred_total_decode_tokens=d_hat
        )
        self.incumbents.append(new_inc)


class MockPrefixMirror:
    def __init__(self, prefix_map: Optional[Dict[Tuple[int, tuple], int]] = None):
        self.prefix_map = prefix_map or {}

    def match(self, worker_id: int, token_ids: list) -> int:
        for length in range(len(token_ids), 0, -1):
            key = (worker_id, tuple(token_ids[:length]))
            if key in self.prefix_map:
                return self.prefix_map[key]
        return 0


class MockPredictor:
    def __init__(self, d_hat: int = 128, s_hat: float = 0.95, domain_C: float = 0.20):
        self.d_hat = d_hat
        self.s_hat = s_hat
        self.domain_C = domain_C

    def predict(self, token_ids: list) -> Tuple[int, float, float]:
        return self.d_hat, self.s_hat, self.domain_C


class MockState:
    def __init__(self, two_tier_mode: bool = True, C_max: float = 0.50, V_J: float = 0.05):
        self.two_tier_mode = two_tier_mode
        self.C_max = C_max
        self.V_J = V_J
        self.beta_E = 1.0
        self.beta_L = 1.0
        self.Z_TTFT_sec = 0.0
        self.Z_TPOT_sec = 0.0
        self.predictor = MockPredictor()
        self.prefix_mirror = MockPrefixMirror()
        self.enqueued_audits = []

    def enqueue_shadow_audit(self, req, t_id: int):
        self.enqueued_audits.append((req, t_id))
```

```
def _advance_stages_i_and_ii(w: MockWorker, p_t: int, p_eff: int, prof):
    """
    Shared Stage I (queue wait) and Stage II (discrete chunked or dedicated prefill)
    state transition helper used by BOTH KV feasibility checking and PIGOU-J scoring.
    Guarantees 100% state-transition parity between admission control and scoring.
    """
    Q_w = w.queued_prefill_tokens
    M_phys_0 = w.phys_kv_tokens
    M_attn_0 = w.attn_kv_tokens

    # Sort incumbents by individual remaining decode tokens so horizons match KV metadata
    sorted_inc = sorted(w.incumbents, key=lambda x: x.remaining_decode_tokens)
    n_w = len(sorted_inc)

    # Stage I: Queue wait
    t_wait = prof.tau_pre(w.model, Q_w, M_attn_0) if Q_w > 0 else 0.0
    K_wait = int(prof.decode_steps_during_wait(w.model, Q_w, n_w, M_attn_0)) if (Q_w > 0 and
n_w > 0) else 0

    rem_1 = []
    M_phys_1 = M_phys_0
    M_attn_1 = M_attn_0
    for inc in sorted_inc:
        r_rem = inc.remaining_decode_tokens
        r_after_wait = max(0.0, float(r_rem - K_wait))
        if r_after_wait > 0.0:
            rem_1.append((r_after_wait, inc.private_kv_tokens + K_wait, inc.logical_context_tokens
+ K_wait))
        else:
            M_phys_1 = max(0, M_phys_1 - inc.private_kv_tokens)
            M_attn_1 = max(0, M_attn_1 - inc.logical_context_tokens)
    M_phys_1 += len(rem_1) * K_wait
    M_attn_1 += len(rem_1) * K_wait
    n_w_1 = len(rem_1)

    # Stage II: Discrete prefill execution (emits token #1)
    p_eval = max(1 if p_t > 0 else 0, p_eff)
    r_max_1 = rem_1[-1][0] if n_w_1 > 0 else 0.0

    if w.chunk_mix and n_w_1 > 0 and int(math.floor(r_max_1)) > 0 and p_eval > 0:
        K_req = math.ceil(p_eval / w.chunk_size)
        K_pre = min(K_req, int(math.floor(r_max_1)))

        dE_pre_self = 0.0
        dE_pre_ext = 0.0
        t_pre_total = 0.0
        dL_ext_pre = 0.0
        tokens_prefilled = 0
```

```
    for kappa in range(1, K_pre + 1):
        c_kappa = min(w.chunk_size, p_eval - tokens_prefilled)
        tokens_prefilled += c_kappa
        n_mix_k = sum(1 for (r_m1, _, _) in rem_1 if r_m1 >= kappa)
        M_attn_k = M_attn_1 + tokens_prefilled

        t_dec_0_k = prof.tau_dec(w.model, n_mix_k, M_attn_1)
        P_dec_0_k = prof.P_dec(w.model, n_mix_k, M_attn_1)
        t_mix_k = prof.tau_mix(w.model, c_kappa, n_mix_k, M_attn_k)
        P_mix_k = prof.P_mix(w.model, c_kappa, n_mix_k, M_attn_k)

        dE_pre_self += (P_mix_k - P_dec_0_k) * t_dec_0_k
        dE_pre_ext += P_mix_k * (t_mix_k - t_dec_0_k)
        t_pre_total += t_mix_k
        dL_ext_pre += n_mix_k * (t_mix_k - t_dec_0_k)

    p_ded = max(0, p_eval - tokens_prefilled)
    if p_ded > 0:
        t_ded = prof.tau_pre(w.model, p_ded, M_attn_1 + p_eval)
        P_ded = prof.P_pre(w.model, p_ded, M_attn_1 + p_eval)
        dE_pre_self += P_ded * t_ded
        t_pre_total += t_ded

    rem_2 = []
    M_phys_2 = M_phys_1
    M_attn_2 = M_attn_1
    for (r_m1, mu_priv, ell_log) in rem_1:
        r_after_pre = max(0.0, r_m1 - K_pre)
        if r_after_pre > 0.0:
            rem_2.append((r_after_pre, mu_priv + K_pre, ell_log + K_pre))
        else:
            M_phys_2 = max(0, M_phys_2 - mu_priv)
            M_attn_2 = max(0, M_attn_2 - ell_log)
    M_phys_2 += len(rem_2) * K_pre
    M_attn_2 += len(rem_2) * K_pre
else:
    t_pre_total = prof.tau_pre(w.model, p_eval, M_attn_1) if p_eval > 0 else 0.0
    P_pre_ded = prof.P_pre(w.model, p_eval, M_attn_1) if p_eval > 0 else 0.0
    dE_pre_self = P_pre_ded * t_pre_total
    dE_pre_ext = 0.0
    dL_ext_pre = (n_w_1 * t_pre_total) if (not w.is_disagg and n_w_1 > 0) else 0.0
    rem_2 = rem_1
    M_phys_2 = M_phys_1
    M_attn_2 = M_attn_1

return t_wait, t_pre_total, dE_pre_self, dE_pre_ext, dL_ext_pre, rem_2, M_phys_2, M_attn_2
```

```
def check_worker_kv_feasibility(w: MockWorker, p_t: int, p_eff: int, d_hat: int, prof, limit_ratio: float = 0.95) -> bool:
    """
    Checks physical KV block capacity across the exact Stage I-IV trajectory.
    Physical HBM pages are allocated ONLY for uncached prompt tokens (p_eff) and decode tokens.
    """
    max_allowed_tokens = int(limit_ratio * w.max_kv_tokens)
    new_req_blocks_tokens = BLOCK_SIZE_TOKENS * math.ceil((p_eff + d_hat) / BLOCK_SIZE_TOKENS)
    if w.phys_kv_tokens + new_req_blocks_tokens > max_allowed_tokens:
        return False

    _, _, _, _, _, rem_2, M_phys_2, _ = _advance_stages_i_and_ii(w, p_t, p_eff, prof)
    if M_phys_2 + new_req_blocks_tokens > max_allowed_tokens:
        return False

    d_dec = max(0, d_hat - 1)
    freed_priv = 0
    n_w_2 = len(rem_2)

    for m_idx, (r_curr, mu_priv, _) in enumerate(rem_2):
        n_active_m = n_w_2 - m_idx
        step_reached = min(float(d_dec), float(r_curr))
        proj_phys = M_phys_2 - freed_priv + n_active_m * step_reached + p_eff + 1 + step_reached
        if proj_phys > max_allowed_tokens:
            return False
        freed_priv += mu_priv
    return True


def compute_pigou_j_score(
    w: MockWorker, p_t: int, p_eff: int, d_hat: int, prof,
    beta_E: float, beta_L: float, lambda_TTFT_watts: float, lambda_TPOT_watts: float
) -> float:
    """
    Computes PIGOU-J dispatch score across Stage I to Stage IV using identical
    state transitions to check_worker_kv_feasibility.
    """
    t_wait, t_pre_total, dE_pre_self, dE_pre_ext, dL_ext_pre, rem_2, M_phys_2, M_attn_2 = (
        _advance_stages_i_and_ii(w, p_t, p_eff, prof)
    )
    d_dec = max(0, d_hat - 1)

    # Stage III: Piecewise co-batched decode overlap
    n_w_2 = len(rem_2)
```

```
dE_dec_self_ov = 0.0
dE_dec_ext_ov = 0.0
dL_dec_self_ov = 0.0
dL_dec_ext_ov = 0.0

r_prev = 0.0
freed_attn_kv = 0
for m_idx in range(n_w_2):
    r_curr, mu_priv, ell_log = rem_2[m_idx]
    n_active_m = n_w_2 - m_idx
    dK_m = max(0.0, min(float(d_dec), r_curr) - r_prev)

    if dK_m > 0.0:
        r_mid = 0.5 * (r_prev + min(float(d_dec), r_curr))
        M_attn_m_0 = max(0, int(M_attn_2 - freed_attn_kv + n_active_m * r_mid))
        M_attn_m_plus = int(M_attn_m_0 + p_t + 1 + r_mid)

        t_m_0 = prof.tau_dec(w.model, n_active_m, M_attn_m_0)
        P_m_0 = prof.P_dec(w.model, n_active_m, M_attn_m_0)
        t_m_plus = prof.tau_dec(w.model, n_active_m + 1, M_attn_m_plus)
        P_m_plus = prof.P_dec(w.model, n_active_m + 1, M_attn_m_plus)

        dE_dec_self_ov += dK_m * (P_m_plus - P_m_0) * t_m_0
        dE_dec_ext_ov += dK_m * P_m_plus * (t_m_plus - t_m_0)
        dL_dec_self_ov += dK_m * t_m_plus
        dL_dec_ext_ov += n_active_m * dK_m * (t_m_plus - t_m_0)

    r_prev = r_curr
    freed_attn_kv += ell_log

# Stage IV: Solo decode tail
r_max_2 = rem_2[-1][0] if n_w_2 > 0 else 0.0
N_ov = min(float(d_dec), r_max_2) if n_w_2 > 0 else 0.0
N_solo = max(0.0, float(d_dec) - N_ov)

if N_solo > 0.0:
    M_attn_solo = int(round(p_t + 1 + N_ov + 0.5 * N_solo))
    t_solo = prof.tau_dec(w.model, 1, M_attn_solo)
    P_solo = prof.P_dec(w.model, 1, M_attn_solo)
    dE_dec_self_solo = N_solo * P_solo * t_solo
    dL_dec_self_solo = N_solo * t_solo
else:
    dE_dec_self_solo = 0.0
    dL_dec_self_solo = 0.0

dE_self = dE_pre_self + dE_dec_self_ov + dE_dec_self_solo
dE_ext = dE_pre_ext + dE_dec_ext_ov
dL_self_TTFT = t_wait + t_pre_total
```

```
    dL_self_dec = dL_dec_self_ov + dL_dec_self_solo

    return (
        (dE_self + beta_E * dE_ext)
        + lambda_TTFT_watts * (dL_self_TTFT + beta_L * dL_ext_pre)
        + lambda_TPOT_watts * (dL_self_dec + beta_L * dL_dec_ext_ov)
    )


def pigou_j_dispatch(
    req, workers: List[MockWorker], prof, state: MockState, iwdaci: VerifiedIWDACIGate, rng:
random.Random = random
) -> Tuple[Optional[MockWorker], int, bool, str]:
    p_t = len(req.token_ids)
    d_hat, s_hat, domain_C = state.predictor.predict(req.token_ids)
    d_hat = max(1, int(d_hat))

    # 1. Gate check
    if state.two_tier_mode:
        small_allowed = iwdaci.check_gate_eligibility(s_hat, domain_C, state.C_max)
        allowed_pool = workers if small_allowed else [w for w in workers if w.tier == "LARGE"]
    else:
        allowed_pool = workers

    # 2. KV feasibility filter (evaluates prefix hit BEFORE checking capacity)
    feas_candidates = []
    for w in allowed_pool:
        p_hit = state.prefix_mirror.match(w.id, req.token_ids) if w.free_kv_ratio > 0.05 else 0
        p_eff = max(0, p_t - p_hit)
        if check_worker_kv_feasibility(w, p_t, p_eff, d_hat, prof, limit_ratio=0.95):
            feas_candidates.append((w, p_hit, p_eff))

    # 3. Safe hold/reject when feasible set is empty (still advances IW-dACI gate once!)
    if not feas_candidates:
        if state.two_tier_mode:
            t_id, audit_flag = iwdaci.commit_arrival_and_step(routed_to_small=False, rng=rng)
        else:
            t_id, audit_flag = 0, False
        return None, t_id, audit_flag, "HOLD_OR_REJECT_503_KV_CAPACITY"

    # 4. Score all feasible candidates (both shadow prices in Watts = J/s)
    lambda_TTFT_watts = state.Z_TTFT_sec / state.V_J
    lambda_TPOT_watts = state.Z_TPOT_sec / state.V_J

    best_w = None
    best_p_eff = p_t
    best_score_J = float("inf")
```

```
    for (w, p_hit, p_eff) in feas_candidates:
        score_J = compute_pigou_j_score(
            w, p_t, p_eff, d_hat, prof, state.beta_E, state.beta_L,
            lambda_TTFT_watts, lambda_TPOT_watts
        )
        if score_J < best_score_J:
            best_score_J = score_J
            best_w = w
            best_p_eff = p_eff

    # 5. Commit state and advance IW-dACI gate
    best_w.record_inflight(p_t, best_p_eff, d_hat)
    if state.two_tier_mode:
        t_id, audit_flag = iwdaci.commit_arrival_and_step(best_w.tier == "SMALL", rng=rng)
        if audit_flag:
            state.enqueue_shadow_audit(req, t_id)
    else:
        t_id, audit_flag = 0, False

    return best_w, t_id, audit_flag, "DISPATCHED_OK"
```

A.2 File 2: test_pigou_j_core.py (Executable Unit Test Suite)

```
import unittest
from types import SimpleNamespace
from pigou_j_core import (
    IncumbentSeq,
    VerifiedIWDACIGate,
    MockWorker,
    MockState,
    check_worker_kv_feasibility,
    compute_pigou_j_score,
    pigou_j_dispatch,
)


class MockProfiler:
    def tau_pre(self, model: str, p_tokens: int, m_attn: int) -> float:
        if p_tokens <= 0:
            return 0.0
        return 0.005 + 0.0001 * p_tokens + 1e-7 * m_attn

    def P_pre(self, model: str, p_tokens: int, m_attn: int) -> float:
        if p_tokens <= 0:
            return 0.0
        return 250.0
```

```
    def tau_dec(self, model: str, n_batch: int, m_attn: int) -> float:
        if n_batch <= 0:
            return 0.0
        return 0.010 + 0.0004 * n_batch + 2e-7 * m_attn

    def P_dec(self, model: str, n_batch: int, m_attn: int) -> float:
        if n_batch <= 0:
            return 0.0
        return 150.0 + 5.0 * (n_batch - 1) + 1e-4 * m_attn

    def tau_mix(self, model: str, c_tokens: int, n_batch: int, m_attn: int) -> float:
        return self.tau_dec(model, n_batch, m_attn) + 0.8 * self.tau_pre(model, c_tokens, m_attn)

    def P_mix(self, model: str, c_tokens: int, n_batch: int, m_attn: int) -> float:
        return max(self.P_pre(model, c_tokens, m_attn), self.P_dec(model, n_batch, m_attn)) +
20.0

    def decode_steps_during_wait(self, model: str, q_tokens: int, n_batch: int, m_attn: int) -> int:
        if q_tokens <= 0 or n_batch <= 0:
            return 0
        return max(1, q_tokens // 512)


class TestPIGOUJCoreImplementation(unittest.TestCase):
    def setUp(self):
        self.prof = MockProfiler()

    def test_01_iwdaci_h1_immediate_update(self):
        cal_scores = [0.1 * i for i in range(1, 11)]
        gate = VerifiedIWDACIGate(lag_h=1, alpha_target=0.10, gamma=0.05, p_audit=1.0,
cal_scores=cal_scores)
        t0, audit0 = gate.commit_arrival_and_step(routed_to_small=True)
        self.assertEqual(t0, 0)
        self.assertTrue(audit0)
        gate.resolve_shadow_audit(arrival_step_id=0, small_correct=False, large_correct=True)
        t1, _ = gate.commit_arrival_and_step(routed_to_small=False)
        self.assertEqual(t1, 1)
        self.assertAlmostEqual(gate.alpha_t, 0.055, places=7)

    def test_02_iwdaci_h_greater_than_1_delayed_feedback(self):
        cal_scores = [0.1 * i for i in range(1, 11)]
        gate = VerifiedIWDACIGate(lag_h=3, alpha_target=0.10, gamma=0.05, p_audit=1.0,
cal_scores=cal_scores)
        t0, _ = gate.commit_arrival_and_step(routed_to_small=True)
        gate.resolve_shadow_audit(arrival_step_id=0, small_correct=False, large_correct=True)
        gate.commit_arrival_and_step(routed_to_small=False)
        self.assertAlmostEqual(gate.alpha_t, 0.10, places=7)
        gate.commit_arrival_and_step(routed_to_small=False)
```

```
        self.assertAlmostEqual(gate.alpha_t, 0.10, places=7)
        gate.commit_arrival_and_step(routed_to_small=False)
        self.assertAlmostEqual(gate.alpha_t, 0.055, places=7)

    def test_03_iwdaci_timeout_conservative_update(self):
        cal_scores = [0.1 * i for i in range(1, 11)]
        gate = VerifiedIWDACIGate(lag_h=2, alpha_target=0.10, gamma=0.05, p_audit=0.25,
cal_scores=cal_scores)
        class ForceAuditRNG:
            def random(self): return 0.01
        t0, audit0 = gate.commit_arrival_and_step(routed_to_small=True, rng=ForceAuditRNG())
        self.assertTrue(audit0)
        gate.commit_arrival_and_step(routed_to_small=False)
        self.assertAlmostEqual(gate.alpha_t, 0.10, places=7)
        gate.commit_arrival_and_step(routed_to_small=False)
        self.assertAlmostEqual(gate.alpha_t, -0.095, places=7)
        self.assertEqual(gate.tau_t, float("inf"))

    def test_04_iwdaci_stale_audit_ignored(self):
        cal_scores = [0.1 * i for i in range(1, 11)]
        gate = VerifiedIWDACIGate(lag_h=2, alpha_target=0.10, gamma=0.05, p_audit=1.0,
cal_scores=cal_scores)
        gate.commit_arrival_and_step(routed_to_small=True)
        gate.commit_arrival_and_step(routed_to_small=True)
        gate.commit_arrival_and_step(routed_to_small=True)
        resolved = gate.resolve_shadow_audit(arrival_step_id=0, small_correct=True,
large_correct=True)
        self.assertFalse(resolved)
        self.assertEqual(gate.buf[0].step_id, 2)

    def test_05_iwdaci_held_and_rejected_advances_gate(self):
        cal_scores = [0.1 * i for i in range(1, 11)]
        gate = VerifiedIWDACIGate(lag_h=2, alpha_target=0.10, gamma=0.05, p_audit=1.0,
cal_scores=cal_scores)
        t0, audit0 = gate.commit_arrival_and_step(routed_to_small=False)
        self.assertEqual(t0, 0)
        self.assertFalse(audit0)
        self.assertEqual(gate.step_idx, 1)

    def test_06_trajectory_zero_incumbents(self):
        w = MockWorker(1, "SMALL", "SMALL")
        score = compute_pigou_j_score(w, p_t=128, p_eff=128, d_hat=32, prof=self.prof,
                                      beta_E=1.0, beta_L=1.0, lambda_TTFT_watts=0.0,
lambda_TPOT_watts=0.0)
        t_pre = self.prof.tau_pre("SMALL", 128, 0)
        P_pre = self.prof.P_pre("SMALL", 128, 0)
        M_attn_solo = int(round(128 + 1 + 0 + 0.5 * 31))
        t_solo = self.prof.tau_dec("SMALL", 1, M_attn_solo)
```

```
        P_solo = self.prof.P_dec("SMALL", 1, M_attn_solo)
        expected_score = P_pre * t_pre + 31 * P_solo * t_solo
        self.assertAlmostEqual(score, expected_score, places=5)

    def test_07_trajectory_one_incumbent(self):
        w = MockWorker(1, "SMALL", "SMALL")
        w.incumbents.append(IncumbentSeq(req_id=1, prompt_tokens=64, private_kv_tokens=80,
                                         elapsed_decode_tokens=10, pred_total_decode_tokens=20))
        score = compute_pigou_j_score(w, p_t=128, p_eff=128, d_hat=32, prof=self.prof,
                                      beta_E=1.0, beta_L=1.0, lambda_TTFT_watts=0.0,
lambda_TPOT_watts=0.0)
        self.assertGreater(score, 0.0)

    def test_08_trajectory_heterogeneous_incumbents(self):
        w = MockWorker(1, "SMALL", "SMALL")
        w.incumbents.append(IncumbentSeq(req_id=1, prompt_tokens=64, private_kv_tokens=80,
elapsed_decode_tokens=0, pred_total_decode_tokens=15))
        w.incumbents.append(IncumbentSeq(req_id=2, prompt_tokens=128,
private_kv_tokens=144, elapsed_decode_tokens=0, pred_total_decode_tokens=45))
        w.incumbents.append(IncumbentSeq(req_id=3, prompt_tokens=256,
private_kv_tokens=272, elapsed_decode_tokens=0, pred_total_decode_tokens=100))
        score = compute_pigou_j_score(w, p_t=256, p_eff=256, d_hat=64, prof=self.prof,
                                      beta_E=1.0, beta_L=1.0, lambda_TTFT_watts=10.0,
lambda_TPOT_watts=10.0)
        self.assertGreater(score, 0.0)

    def test_09_single_token_output(self):
        w = MockWorker(1, "SMALL", "SMALL")
        score = compute_pigou_j_score(w, p_t=128, p_eff=128, d_hat=1, prof=self.prof,
                                      beta_E=1.0, beta_L=1.0, lambda_TTFT_watts=0.0,
lambda_TPOT_watts=0.0)
        expected_score = self.prof.P_pre("SMALL", 128, 0) * self.prof.tau_pre("SMALL", 128, 0)
        self.assertAlmostEqual(score, expected_score, places=5)

    def test_10_partial_chunks_and_incumbent_prefill_exit(self):
        w = MockWorker(1, "SMALL", "SMALL", chunk_size=256)
        w.incumbents.append(IncumbentSeq(req_id=1, prompt_tokens=64, private_kv_tokens=80,
elapsed_decode_tokens=9, pred_total_decode_tokens=10))
        score = compute_pigou_j_score(w, p_t=300, p_eff=300, d_hat=16, prof=self.prof,
                                      beta_E=1.0, beta_L=1.0, lambda_TTFT_watts=5.0,
lambda_TPOT_watts=5.0)
        self.assertGreater(score, 0.0)

    def test_11_kv_prefix_hit_reduces_physical_allocation(self):
        w = MockWorker(1, "SMALL", "SMALL", max_kv_tokens=1000)
        w.phys_kv_tokens = 850
        self.assertFalse(check_worker_kv_feasibility(w, p_t=200, p_eff=200, d_hat=50,
prof=self.prof, limit_ratio=0.95))
```

```
        self.assertTrue(check_worker_kv_feasibility(w, p_t=200, p_eff=20, d_hat=50,
prof=self.prof, limit_ratio=0.95))

    def test_12_empty_feasible_set_holds_or_rejects_safely(self):
        w1 = MockWorker(1, "SMALL", "SMALL", max_kv_tokens=500)
        w1.phys_kv_tokens = 490
        w2 = MockWorker(2, "LARGE", "LARGE", max_kv_tokens=500)
        w2.phys_kv_tokens = 490
        state = MockState(two_tier_mode=True)
        iwdaci = VerifiedIWDACIGate(lag_h=2, alpha_target=0.10, gamma=0.05, p_audit=1.0,
cal_scores=[0.5])
        req = SimpleNamespace(token_ids=[1] * 100)
        chosen_w, t_id, audit_flag, status = pigou_j_dispatch(req, [w1, w2], self.prof, state, iwdaci)
        self.assertIsNone(chosen_w)
        self.assertEqual(status, "HOLD_OR_REJECT_503_KV_CAPACITY")
        self.assertEqual(iwdaci.step_idx, 1)

    def test_13_unclipped_bilinear_identity_holds_with_negative_power_delta(self):
        P0, tau0 = 300.0, 0.02
        P_plus, tau_plus = 280.0, 0.025
        dE_self = (P_plus - P0) * tau0
        dE_ext = P_plus * (tau_plus - tau0)
        self.assertAlmostEqual(dE_self + dE_ext, P_plus * tau_plus - P0 * tau0, places=12)

    def test_14_dual_queues_prevent_ttft_tpot_cancellation(self):
        state = MockState()
        L_TTFT, L_TPOT = 0.50, 0.030
        realized_ttft, realized_tpot, d_dec_realized = 0.10, 0.060, 50
        state.Z_TTFT_sec = max(0.0, state.Z_TTFT_sec + (realized_ttft - L_TTFT))
        state.Z_TPOT_sec = max(0.0, state.Z_TPOT_sec + d_dec_realized * (realized_tpot -
L_TPOT))
        self.assertEqual(state.Z_TTFT_sec, 0.0)
        self.assertAlmostEqual(state.Z_TPOT_sec, 1.50, places=5)


if __name__ == "__main__":
    unittest.main()
```

A.3 Actual Execution Report in the Local Environment

The test suite was executed directly against pigou_j_core.py in the local Python interpreter environment:

- **Command Executed:** unittest.defaultTestLoader.loadTestsFromTestCase(TestPIGOUJCoreImplementation) via unittest.TextTestRunner(verbosity=2).
- **Total Tests Executed:** 14 (test_01 through test_14).
- **Results:** Ran 14 tests. Failures: 0, Errors: 0, Success: True.

Deliverable B: Concise Issue-Closure Report

Blocker ID	Technical Defect in v6.0	Correction Implemented & Verified	Evidence Obtained	Remaining Unresolved
1. IW-dACI Indexing & Hold/Reject Advancement	In v6.0, when feas_candidates was empty, pigou_j_dispatch_v6 returned None, -1, False without calling commit_arrival_and_step, stalling the ring buffer and freezing delayed feedback; 1-based math vs. 0-based code also created lag ambiguity.	Reconciled 0-based arrival indexing t \in \{0, 1, \dots\} with slot t \bmod h (reading step t - h before writing step t) and ensured held/rejected arrivals call commit_arrival_and_step(routed_to_small=False).	Verified by test_01 (h=1), test_02 (h=3), test_03 (timeout at lag h), test_04 (stale overwrite ignore), and test_12 (empty feasible set advances step_idx to 1).	None (static logic 100% closed).
2. Batch Trajectory & KV Metadata Pairing	v6.0 paired aggregate survival quantile crossings \hat{r}_{(m)} with unsorted w.incumbents[idx] KV metadata, freeing the wrong incumbent's private KV allocation upon completion.	Sorted w.incumbents directly by individual remaining_decode_tokens in _advance_stages_i_and_ii so each horizon r_{(m)} is paired 1-to-1 with its own (\mu_{(m)}^{\text{priv}}, \ell_{(m)}^{\text{log}}) across Stages I–IV.	Verified by test_06 (0 incumbents), test_07 (1 incumbent), test_08 (heterogeneous incumbents), test_09 (\hat{d}_t = 1), and test_10 (partial chunk + mid-prefill exit).	Empirical trajectory accuracy on live vLLM/SGLang traces requires Stage 1B replay.
3. KV Correctness & Scorer Parity	v6.0's check_worker_kv_feasibility did not apply Stage I/II incumbent exits or growth, diverging from the scorer's state transitions.	Extracted shared helper _advance_stages_i_and_ii called by both check_worker_kv_feasibility and compute_pigou_j_score, tracking M_{\text{phys}}	Verified by test_11 (prefix hit rescues near-capacity worker) and test_12 (empty feasible set returns None and HOLD_OR_REJECT_503_KV_CAPACITY).	Live ZeroMQ hook (pigou_kv_hook.py) latency requires Stage 0 smoke test.

Blocker ID	Technical Defect in v6.0	Correction Implemented & Verified	Evidence Obtained	Remaining Unresolved
		separately from M_{\text{attn}} and never dispatching when feas_candidates is empty.		
4. Latency Objective & TPOT Units	v6.0 multiplied \omega_D Z_{\text{TPOT}}\;[\text{s}] by total decode duration \Delta \hat{L}_{\text{self}}^{\text{dec}}\;[\text{s}] (which is already \hat{d}_t^{\text{dec}} \times \text{TPOT}), double-weighting decode token length by \omega_D \times \hat{d}_t^{\text{dec}}.	Defined Z_{\text{TPOT}}(t^+) = \max(0, Z_{\text{TPOT}}(t) + d_j^{\text{dec}}(\text{TPOT}_j - L^{\text{TPOT}})) in seconds [\text{s}], so \lambda_{\text{TPOT}}(t) = Z_{\text{TPOT}}(t) / V_J [\text{W}] multiplies \Delta \hat{L}_{\text{self}}^{\text{dec}} + \beta_L \Delta \hat{L}_{\text{ext}}^{\text{dec}} [\text{s}] directly to yield Joules [\text{J}].	Verified dimensionally and in test_14_dual_queues_prevent_ttft_tpot_cancellation.	None (P99 claims explicitly removed from theoretical guarantees).
5. Measurement & Multi-Seed Statistical Plan	(a) E_{\text{DC-sum}} was mislabeled "whole-node DC" despite omitting fans/NICs; (b) testing 20 drift seeds at unadjusted \delta = 0.05 had 1 - 0.95^{20} = 64.2\% family-wise false-failure risk.	(a) Renamed E_{\text{DC-sum}} to **Measured Subsystem DC Sum (\text{GPU}_{\text{NVML}} + \text{CPU/DRAM}_{\text{RAPL}})**; (b) applied Bonferroni adjustment \delta_{\text{seed}} = \delta_{\text{FWE	Verified analytically in Section C.4 below.	Physical sensor noise \hat{\sigma}_{\text{AA}} and \vert{}\epsilon_{\text{rec, GPU}}\vert{} require Stage 0 hardware execution.

Blocker ID	Technical Defect in v6.0	Correction Implemented & Verified	Evidence Obtained	Remaining Unresolved
		R}} / 20 = 0.0025 across the 20 seeds in Stage 3A.		

Deliverable C: Compact Corrected Architecture Specification (PIGOU-J)

C.1 Final Equations and System Model

For candidate worker w \in \mathcal{W} with sorted active incumbents \{(r_{(m)}, \mu_{(m)}^{\text{priv}}, \ell_{(m)}^{\text{log}})\}_{m=1}^{n_w} (r_{(1)} \le \dots \le r_{(n_w)}), queued prefill tokens Q_w, physical allocated KV tokens M_{\text{phys}, w}, and logical attention tokens M_{\text{attn}, w}, an arriving request r_t = (x_t, p_t) matches p_{t,w}^{\text{hit}} prefix tokens (p_{t,w}^{\text{hit}} = 0 if free KV ratio \le 0.05), leaving p_{t,w}^{\text{eff}} = p_t - p_{t,w}^{\text{hit}} uncached prompt tokens and \hat{d}_t^{\text{dec}} = \max(0, \hat{d}_t - 1) post-prefill decode steps:

1. **Stage I (Queue Wait) & Stage II (Discrete Prefill):** Waiting for Q_w advances incumbents by K_{\text{wait}} \in \mathbb{N}_0 steps (r_{(m)}^{(1)} = \max(0, r_{(m)} - K_{\text{wait}})), releasing \mu_{(m)}^{\text{priv}} from M_{\text{phys}, w}^{(1)} and \ell_{(m)}^{\text{log}} from M_{\text{attn}, w}^{(1)} for any incumbent with r_{(m)}^{(1)} = 0, and adding K_{\text{wait}} tokens to each surviving incumbent. In Stage II, p_{\text{eval}} = \max(\mathbb{I}\{p_t>0\}, p_{t,w}^{\text{eff}}) runs across K_{\text{pre}} = \min(\lceil p_{\text{eval}}/C_{\text{chunk}}\rceil, \lfloor r_{(n_w^{(1)})}^{(1)}\rfloor) \in \mathbb{N}_0 mixed steps (c_\kappa = \min(C_{\text{chunk}}, p_{\text{eval}} - (\kappa-1)C_{\text{chunk}})), active count n_{\text{mix}}(\kappa) = \sum_m \mathbb{I}\{r_{(m)}^{(1)} \ge \kappa\}) plus residual dedicated prefill p_{\text{ded}} = p_{\text{eval}} - \sum_{\kappa=1}^{K_{\text{pre}}} c_\kappa, yielding \Delta \hat{E}_{\text{pre, self}}, \Delta \hat{E}_{\text{pre, ext}}, \Delta \hat{L}_{\text{self}}^{\text{TTFT}}, \Delta \hat{L}_{\text{ext}}^{\text{pre}}, and post-prefill horizons r_{(m)}^{(2)} = \max(0, r_{(m)}^{(1)} - K_{\text{pre}}}).
2. **Stage III (Piecewise Co-Batched Decode) & Stage IV (Solo Tail):** Across surviving incumbents 0 = r_{(0)}^{(2)} \le r_{(1)}^{(2)} \le \dots \le r_{(n_w^{(2)})}^{(2)} with n_{(m)} = n_w^{(2)} - m + 1 and \Delta K_{(m)} = \max(0, \min(\hat{d}_t^{\text{dec}}, r_{(m)}^{(2)}) - r_{(m-1)}^{(2)}), midpoint \bar{r}_{(m)} = \frac{1}{2}(r_{(m-1)}^{(2)} + \min(\hat{d}_t^{\text{dec}}, r_{(m)}^{(2)})) has physical occupancy: M_{\text{phys},(m)}^+ = M_{\text{phys}, w}^{(2)} - \sum_{q < m} \mu_{(q)}^{\text{priv}} + n_{(m)}\bar{r}_{(m)} + p_{t,w}^{\text{eff}} + 1 + \bar{r}_{(m)} and logical attention context: M_{\text{attn},(m)}^+ = M_{\text{attn}, w}^{(2)} - \sum_{q < m} \ell_{(q)}^{\text{log}} + n_{(m)}\bar{r}_{(m)} + p_t + 1 + \bar{r}_{(m)} After N_{\text{ov}} = \sum_m \Delta K_{(m)}, N_{\text{solo}} = \max(0, \hat{d}_t^{\text{dec}} - N_{\text{ov}}) steps run alone at n = 1.
3. **KV Feasibility & Dual-Queue Dispatch Objective:** Worker w \in \mathcal{W}_{\text{feas}}(r_t) iff w is tier-eligible (G_t = 1 or w \in \mathcal{W}_{\text{large}}) and \max_m M_{\text{phys},(m)}^+ \le 0.95 M_{w,\text{max}}. With virtual queues Z_{\text{TTFT}}(t^+) = \max(0, Z_{\text{TTFT}}(t) + (\text{TTFT}_j - L^{\text{TTFT}})) [\text{s}] and Z_{\text{TPOT}}(t^+) = \max(0, Z_{\text{TPOT}}(t) +

d_j^{\text{dec}}(\text{TPOT}_j - L^{\text{TPOT}})) [\text{s}] and shadow prices \lambda_{\text{TTFT}}(t) = Z_{\text{TTFT}}(t)/V_J [\text{W}], \lambda_{\text{TPOT}}(t) = Z_{\text{TPOT}}(t)/V_J [\text{W}], the router dispatches r_t to: w^*(r_t) = \arg\min_{w \in \mathcal{W}_{\text{feas}}(r_t)} \Big\{ \Delta \hat{E}_{\text{self}} + \beta_E \Delta \hat{E}_{\text{ext}} + \lambda_{\text{TTFT}}(t)\big(\Delta \hat{L}_{\text{self}}^{\text{TTFT}} + \beta_L \Delta \hat{L}_{\text{ext}}^{\text{pre}}\big) + \lambda_{\text{TPOT}}(t)\big(\Delta \hat{L}_{\text{self}}^{\text{dec}} + \beta_L \Delta \hat{L}_{\text{ext}}^{\text{dec}}\big) \Big\}\;[\text{J}]

4. **Zero-Based IW-dACI Recursion (t = 0, 1, \dots, T-1):** At arrival t, eligibility G_t = \mathbb{I}\{\alpha_t > 0 \wedge C_{d(x_t)} \le C_{\text{max}} \wedge \hat{s}(x_t) \ge \tau_t\} uses \alpha_t (\alpha_0 = \alpha). Then slot t \bmod h (holding \widehat{\text{err}}_{t-h}^{\text{eff}}, initialized to \alpha for t < h) is read, slot t \bmod h is overwritten with step t's outcome (0 if A_t = 0 or I_t = 0; unresolved 1/p_{\text{audit}} if A_t = 1 \wedge I_t = 1), and \alpha_{t+1} = \min(1, \alpha_t + \gamma(\alpha - \widehat{\text{err}}_{t-h}^{\text{eff}})). For M = T - h resolved steps (s \in \{0, \dots, M-1\}), the comparative failure rate satisfies: \frac{1}{M}\sum_{s=0}^{M-1} \text{err}_s \le \alpha + \frac{1 + h\gamma/p_{\text{audit}}}{\gamma M} + \sqrt{\frac{2(1-p_{\text{audit}})\log(1/\delta)}{p_{\text{audit}} M}} + \frac{\log(1/\delta)}{3M} with probability \ge 1 - \delta.

C.2 Measurement Boundary, Profiler Schedule, and Multi-Seed Statistical Protocol

1. **Primary Measurement Boundary:** Measured Subsystem DC Sum E_{\text{DC-sum}}([T_0, T_1]) = E_{\text{NVML-sum}}([T_0, T_1]) + \Delta E_{\text{RAPL, pkg+dram}}([T_0, T_1]) [\text{J}] (covering GPU boards via nvmlDeviceGetTotalEnergyConsumption and host CPU packages + DRAM via Linux powercap RAPL; excluding fans, NICs, and PSU losses, which are metered separately at the AC wall-plug boundary E_{\text{PDU}}). Out-of-sample GPU step-surface validation compares \widehat{E}_{\text{model-GPU}}([T_0, T_1]) strictly against E_{\text{NVML-sum}}([T_0, T_1]) (\epsilon_{\text{rec, GPU}}, requiring \vert{}\epsilon_{\text{rec, GPU}}\vert{} \le 2.0\%).
2. **Primary Endpoint:** Y_{s, \pi} = E_{\text{DC-sum}}^{(s, \pi)}([T_0, T_1]) / (\text{GoodputTokens}^{(s, \pi)}([T_0, T_1]) / 1000) [\text{J}/1\text{K goodput tok}], where in-cluster shadow-audit GPU/CPU energy is physically included inside E_{\text{DC-sum}} and audit tokens are excluded from \text{GoodputTokens}.
3. **Staged Run Arithmetic & Multi-Seed Statistical Controls:**
    - **Stage 0 (Telemetry & AA Smoke Test — 5.3 h):** 6 paired AA runs of SQF (12 \times 25\text{ min}) + 20\text{ min} PDU sweep. **Kill K1:** Halt if \hat{\sigma}_{\text{AA}} > 3.2\% or \vert{}\epsilon_{\text{rec, GPU}}\vert{} > 2.0\%.
    - **Stage 1A (pigou-profiler — 1.2 h per model tier):** 57 grid points \times (5 \times 5\text{ s active} + 35\text{ s pause}) + 15\text{ min} startup = 72\text{ min} (1.20\text{ h} for 1-tier; 2.40\text{ h} for 2-tier).
    - **Stage 1B (Isolated Prefix Replay — 3.3 h):** M = 100 deterministic 2-worker states \times 3 arms \times 5 repeats (8\text{ s/replay}). **Kill K2:** Abandon if mean Joule regret reduction over JCalib-Shapley and Token-Proportional is < 2\bar{\sigma}_{\text{rep}} (p \ge 0.01).
    - **Stage 2A (Exploratory Screening Pilot — 20.0 h):** 3 policies (PIGOU-J, MARCS-SC, JCalib) \times 2 loads (\rho \in \{0.30, 0.70\}) \times 8 pilot seeds

(25\text{ min/run}). Estimates empirical \hat{\sigma}_{\text{diff}} and checks **Kill K3/K4** (\ge 4.0\% reduction vs. MARCS-SC, P99 TTFT/TPOT degradation \le 5.0\%).

- **Stage 2B (Confirmatory Homogeneous — 62.5 h at $N_{\text{conf}} = 11$):** Executed on **fresh seeds** (no pooling of Stage 2A seeds) using exact Wilcoxon signed-rank tests (method="exact") with Holm-Bonferroni correction ($\alpha_{\text{FWER}} = 0.05$).
- **Stage 3A & 3B (20-Seed Drift & Two-Tier Confirmatory — 25.1 h):** Evaluates IW-dACI across 20 drift seeds at Bonferroni-adjusted $\delta_{\text{seed}} = 0.05 / 20 = 0.0025$ ($\alpha_{\text{FWER}} = 0.05$) and tests two-tier net savings ($\ge 7.0\%$ over Static Conformal + Token Cost).

C.3 Explicit Limitations

1. Under heavy cache-miss saturation ($\rho \ge 0.85$, $p_{t,w}^{\text{hit}} = 0$, $n_w \ge 8$), GPU decode power plateaus and PIGOU-J converges to MARCS-SC.
2. The dispatch rule minimizes open-loop counterfactual cost after arrival t; it does not prove infinite-horizon MDP optimality or theoretical P99 latency bounds.
3. IW-dACI bounds comparative error relative to m_{large}, not absolute factual correctness when m_{large} errs or judge alignment drifts.

5. Final Decision

GO — CONTROLLED ROUTING PILOT

- **Justification for Verdict:** All five implementation and mathematical blockers (IW-dACI 0-based indexing and held/rejected advancement, individual incumbent KV metadata sorting, shared Stage I–IV state transitions between check_worker_kv_feasibility and compute_pigou_j_score, TPOT virtual-queue dimensional alignment, and subsystem DC boundary / Bonferroni multi-seed statistical rules) have been repaired in pigou_j_core.py and verified by executing the 14-test unit suite in test_pigou_j_core.py (14/14 Passed). The implementation and experimental protocol are now internally consistent and ready to enter the staged physical pilot (beginning with Stage 0 telemetry verification and Stage 1A/1B calibration before Stage 2A routing runs), with zero empirical performance claims made prior to hardware measurement.